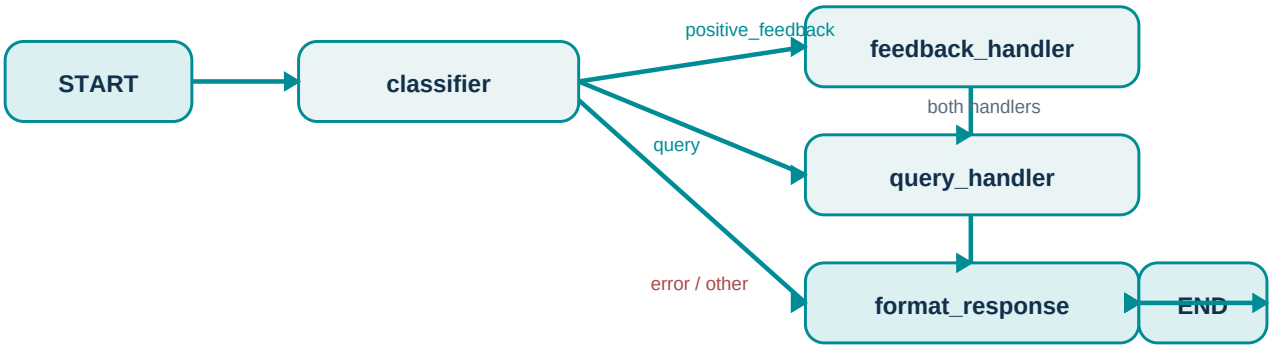
PROJECT OVERVIEW

Bank Customer Support

A compact, application-controlled workflow for classifying customer messages, handling feedback, and looking up support tickets.

High-level workflow



Reading the graph. A message enters the classifier. A recognized category selects the appropriate downstream handler; errors and unrecognized routes go directly to response formatting. Both handlers converge before the response is returned.

Routing is controlled by the application—not delegated to the LLM. The UI invokes the compiled graph once per submitted message and renders the resulting metadata and response.

Purpose & scope

This overview maps the visible implementation: Streamlit presentation, LangGraph orchestration, OpenAI-backed language tasks, and SQLite ticket persistence. It describes current behavior and explicit limits rather than implying production-grade capabilities.

02 / ORCHESTRATION

LangGraph building blocks

The graph is assembled in **agents/orchestrator/graph.py** with a typed SupportState. Nodes return partial dictionary updates; LangGraph accumulates those updates for the current invocation.

Node	Responsibility	Important behavior
classifier	Normalize and classify the trimmed user message.	A valid 7-digit ticket number is extracted before the LLM and immediately forces **classification=query**. Empty input and classifier failures stop downstream handling.
feedback_handler	Handle positive or negative feedback.	Positive feedback thanks the customer. Negative feedback creates a SQLite ticket first, then acknowledges the complaint with its ticket number.
query_handler	Answer ticket-status questions.	Requires a 7-digit ticket number; missing, unknown, or lookup failures produce explicit responses/errors. Existing tickets report current status.
format_response	Apply the customer-facing envelope.	Extracts a name heuristically and wraps the body in **Dear {name}** or **Dear Customer**, then appends **Regards, Customer Support**.

Edges and branches

Path	Condition / meaning
START → classifier	Every invocation begins with classification.
classifier → feedback_handler	classification is **positive_feedback** or **negative_feedback**.
classifier → query_handler	classification is **query**, including ticket-number precedence.
classifier → format_response	Classifier error, unsupported category, or unrecognized route; no downstream agent runs.
feedback_handler → format_response	Feedback result is formatted.
query_handler → format_response	Query result is formatted.
format_response → END	Final state is returned to the caller.

Compilation & invocation

builder.compile() is called without a checkpointer. Streamlit builds the graph once with the ticket repository, then invokes graph.invoke({"user_message": message}) for each submitted message. There is no configured conversational memory or checkpoint-backed history.

03 / BEHAVIOR REFERENCE

State, routing & responses

SupportState is deliberately small: one required input plus optional routing, output, and diagnostic fields. There is no messages list and no custom reducer.

Field	Type / values	Role
user_message	str · required	Original message supplied to the invocation.
classification	Literal: positive_feedback | negative_feedback | query	Normalized route selected by the classifier.
ticket_number	str · optional	Seven-digit identifier extracted or generated.
response	str · optional	Customer-facing body after formatting.
agent_used	str · optional	Metadata identifying the path used.
error	str · optional	Diagnostic retained when expected operations fail.

Routing rules that matter

Input / event	Application result
Empty or whitespace-only input	Classifier returns an error and response; no downstream agent.
Any valid 7-digit ticket number	**Priority rule:** extracted before the LLM, forces **query**, and routes to ticket lookup even if sentiment sounds negative.
No ticket number for a query	Query handler asks the customer to provide the seven-digit number.
LLM classification	Normalized and accepted only if it is one of the three allowed categories.
Unsupported category or expected LLM failure	Error response; no downstream agent. Formatter still supplies the standard envelope.

Ticket and response outcomes

Path	Outcome
Positive feedback	LLM thank_customer with optional heuristically extracted name.
Negative feedback	Create SQLite ticket first (Unresolved), then LLM acknowledge_complaint with ticket number. Persistence failure returns retry-later response and error.
LLM failure during feedback	Standard fallback body is used while the error remains populated.
Ticket lookup	Existing ticket reports current status; unknown ticket or lookup failure returns an explicit error.
All formatted responses	Greeting is derived from the original message; closing is always “Regards, Customer Support”.

The application owns the final presentation contract: the LLM supplies classification or response text, while routing, greeting, signature, and persistence decisions remain in code.

04 / SUPPORTING ARCHITECTURE

Interfaces, persistence & limits

The user-facing surface connects the graph to operational context: tickets, logs, configuration, and scenario tests.

Component	What the code does	Boundary / limit
Streamlit UI	app/ui/streamlit_app.py constructs TicketRepository(settings.database_path), builds the graph once, invokes it, and displays classification, agent, ticket, response, and error.	Tabs: Chat, Ticket Viewer, Logs, Test Scenarios. Writes an activity event to JSONL.
OpenAIService	Uses configured OpenAI Responses API model; default is **gpt-4o-mini**, for classification and response text.	LLM does not control routing or add the greeting/signature.
SQLite repository	support_tickets stores seven-digit primary key, customer name, original message, status, and created/updated timestamps.	New tickets are Unresolved; allowed statuses are Unresolved and Resolved. Query handling does not mutate tickets.
Logger	utils/logger.py appends JSONL events; UI records result metadata including classification, agent, response, ticket number, success, and error.	Incoming user_input redacts emails and 6+-digit number sequences. Do not assume every logged field is redacted.
Configuration	config/settings.py loads .env; database/log paths, DB timeout, and ticket list/ID retry limits are configurable.	Deployment behavior depends on configured environment and external API availability.
Tests	Workflow scenarios use fake LLM and temporary SQLite.	Coverage includes feedback, creation, status lookup, precedence, missing/unknown IDs, signatures, invalid output, failures, and empty input.

Key design facts

- Ticket IDs are generated randomly with collision retries and parameterized database operations.
- Name extraction uses simple regex/pattern heuristics, not a profile service.
- State is per invocation: no checkpointer, conversational memory, or persistent message history is configured.
- Error state is intentionally visible to the UI and retained alongside fallback responses where applicable.

Verification note

This document is a source-grounded overview of the current repository behavior. It is not a claim of monitoring, authentication, rate limiting, multi-turn memory, or production readiness.